Dokumentation des Erstsemesterprojekt (Gruppe 02D (BM3))

Die vorliegende Dokumentation fasst die im Rahmen des Erstsemesterprojekts FB03 durchgeführten Arbeiten, Vorgehensweisen und Ergebnisse der Gruppe BM3 zusammen, und gibt einen strukturierten Überblick über die erarbeiteten Inhalte.

Im Rahmen des Erstsemesterprojekts FB03, welches studiengangübergreifend die ersten Fachsemester der Studiengänge Bachelor Informatik, Bachelor Elektrotechnik und Bachelor Maschienenbau an der Hochschule Niederrhein (Campus Krefeld Süd) einschließt, gilt es folgende Aufgabe in Gruppen aus jeweils vier Studenten zu lösen:

„Die Aufgabe des Projekts ist es, einen Roboter zu bauen und zu programmieren, der mit einem Smartphone ferngesteuert werden kann. Auf einem ca 3x3m großem Spiefeld, das durch eine Trennlinie in zwei gleiche Felder unterteilt ist, werden je 8 Bälle abgelegt, die der Fahr-Roboter möglichst ins gegnerische Feld, besser jedoch gleich ins gegnerische Tor schießt. Die Trenn-Linie darf maximal mit der halben Roboterlänge überfahren werden. Zusätzlich ist das Projekt vollständig zu dokumentieren.“

Gruppenmitglieder und allgemeine Aufgabenverteilung

Bastian Büld, MatrikelNr.: (Bachelor Informatik):
Programmieren

Marius Spans, MatrikelNr.1763476: (Bachelor Elektrotechnik):
Bau & Verkabelung

Markus Sladkowski, MatrikelNr.: 1784606 (Bachelor Informatik):
Dokumentation

Max Wimmer, MatrikelNr.: 1771785 (Bachelor Informatik):
Design und 3D-Druck

Die Aufgaben wurden unter Priorisierung einer weitgehenst parallelen Arbeitsweise und in anbetracht bestehender individueller Fähigkeiten der Gruppenmitglieder verteilt.

Inhaltsverzeichnis

- Gruppenmitglieder und allgemeine Aufgabenverteilung Seite 1
- Zusammenfassung erstes Meeting Seite 3
- Organisation Seite 3
- Bau & Verkabelung (Marius Spans) Seite 4-5
- Design & 3D-Druck (Max Wimmer) Seite 6-8
- Programmieren (Bastian Büld) Seite 9
- Quellen und Hilfsmittel Seite 10-31

    Ergebnis des ersten Gruppenmeetings Seite 11
    Schaltplan V1 Seite 12
    Schaltplan V2 Seite 13
    Schaltplan V3 Seite 14
    Pin Belegungen Seite 15
    Erster und Zweiter Prototyp Seite 16
    Dritter und Vierter Prototyp Seite 17
    Fünfter Prototyp Seite 18
    Source Code Seite 19-29

-Quellen und Hilfsmittel (zweiter Meilenstein) Seite 32-46

    Änderung der Pinbelegung Seite 32
    Schaltplan V4 Seite 33
    Schaltplan V5 Seite 34
    Source Code (Änderungen und Ergänzungen) Seite 35-46

Zusammenfassung erstes Meeting

Das Meeting fand online über einen Videoanruf mit allen Gruppenmitgliedern über die MessengerApp »Discord« statt.

Im Vorfeld haben Bastian und Marius die Fahrzeuge der Vorjahren dokumentiert und der Gruppe während des Meetings in From von Bildmaterial zur Verfügung gestellt. Unter Berücksichtigung dieser visuellen Darstellung konnte sich unsere Gruppe schnell auf ein allgemeines Design des Fahrzeugs einigen. Hauptsächliche Entscheidungsmerkmale waren dabei die technische Umsetzbarkeit im 3D-Druck sowie die Stabilität und Handhabbarkeit beim Bau in relation zum anzunehmenden Endprodukt.

Nachdem eine Einigung über das allgemeine Design erzielt wurde, hat unsere Gruppe verschiedene Schießmechanismen für die Bälle im finalen Spiel ausgearbeitet und verglichen. Dabei wurde schnell deutlich, welche theoretischen Lösungen tatsächlich mit der Designphilosophie kompatibel waren. Ein theoretischer Aufnahme-Mechanismus, mit dem Ziel bis zu drei Bälle innerhalb des Fahrzeuges zu lagern und für den Schießmechanismus vorzubereiten, wurde auf dieselbe Weise konzipiert.

Für die visuelle Ausarbeitung der theoretischen Modelle wurde die Whiteboard Funktion von Discord verwendet (siehe Quellen und Hilfsmittel). Nach mehreren Anläufen die theoretisch ausgearbeiteten Mechanismen visuell zu realisieren wurden noch bestehende Problematiken deutlich. Da diese Problematiken mit der technischen Umsetzung in Verbindung standen haben wir entschieden mehrere Lösungsansätze realitätsnah umzusetzen und, nach einer erfolgreichen Testreihe im dreidimensionalen Raum, die Ergebnisse in einem späteren Meeting als Gruppe zu besprechen und ein mögliches Umdenken für einen optimaleren Lösungsweg zu erwähnen.

Abschließend wurde noch über hypothetische Teile, Anbauten und ähnliche Extras diskutiert, jedoch war sich die Gruppe einig, das die Ergebnisse der geplanten Testreihe erst vorliegen müssen um eine informierte Entscheidung bezüglich weiterer Maßnahmen treffen zu können.

Organisation

(Erster Meilenstein)
Neben den bereits im detail besprochenem Discord Meeting hat sich unsere Gruppe zusätzlich über die MessengerApp »WhatsApp« organisiert und verständigt.
Dokumente und projektrelevante Dateien wurden über ein git.ide3 Repository (siehe Quellen und Hilfsmittel) von den einzelnen Gruppenmitgliedern dem Rest der Gruppe offengelegt und zur Verfügung gestellt.

(zweiter Meilenstein)
Es gab keine größeren Meetings über Discord wie noch zu Anfang, da jedes Gruppenmitglied seine Aufgabe verstand und gewissendlich nachgekommen ist. Generelle Kommunikation fand weiterhin wie im ersten Meilenstein statt.
(Whatsapp, git.ide3 Repository, In-Person treffen an der Hochschule)

Bau & Verkabelung (Marius Spans)

Erster Meilenstein:

Folgende Arbeiten wurden innerhalb der ersten Entwicklungsphase (erster Meilenstein) abgeschlossen:

1. Grundgestell des Wagens zusammengebaut.
    - -Motoren an der Bodenplatte es Gestells befestigt und die Reifen aufgesteckt.
    - -Das Front Rad mit den dazugehörigen Abstandshaltern befestigt.
    - -Platinenhalter fixiert:
        - -Gewinde in die passenden Löcher der Bodenplatte gebohrt und die Stützen des 3D gedruckten Platinenhalters mit gewinde versehen.
        - -Neue Schrauben zum fixieren des Platinenhalters verwendet, so dass dieser stabil auf dem Gestell montiert werden konnte.

2. Aderleitungen an die Motoranschlüsse des Fahrwerks angelötet. (Bild folgt noch)

3. Testaufbau begonnen, Antriebsmotoren an das L9110 Dual-DC Motor-Treiber Board angeschlossen und getest, ob die Motoren angesteuert werden können.
    - -Test war erfolgreich.
    - -Weiter herum experimentiert und die Motoren über die externen Betterien angeschlossen.
    - -GND muss zusätzlich mi dem Microcontroller verbunden sein, sonst gehen die Motoren nicht an.

4. Ersten Schaltplan erstellt, um eine grobe Übersicht von dem Bauteilen zu bekommen und um den weiteren Aufbau zu erleichtern.

Der initiale Schaltplan (Schaltplan V1) wurde mit Bautelen erweitert, welche für die festgelegten Mechanismen benötigt wurden. (Siehe Schaltplan V2)

Schaltplan V2 wurde angepasst, da festgestellt wurde, dass der Servomotor nicht für den eigentlichen Zweck verwendet werden kann. Entsprechen wird vorläufig ein gewöhnlicher DC Motor zum Testen des Konzepts verwendet.
Zusätzlich wurde der Schaltplan optimiert hinsicht der beiden zusätzlichen DC Motoren für den Schießmechanismus, indem diese parallel verschaltet wurden. (Siehe Schaltplan V3)

Die Pinbelegung wurde festgelegt und für das Programmieren bereitgestellt. (Siehe Pinbelegung)

Final wurde noch eine Liste an benötigten Teilen erstellt, um eine Gewährleistung der Umsetzbarkeit sicherzustellen:
- -LN20g Modul für die Ansteureung der beiden Motoren
- -2 Räder (Für den Schießmechanismus)
- -2 zusätzliche DC Motoren (Redundanz) + 1 weiterer DC Motor (Für den Aufnahmemechanismus)

Zweiter Meilenstein:

1. Änderungen und Ergänzungen am Grundgestell des Fahrzeugs:
    - -Sensoren angeschlossen und am Gehäuse fixiert.
    - -Motoren für den Schussmechanismus verdrahtet und das Masse-Potential der Powerbank mit Wago Klemmen zusammengeführt.
    - -Wago Klemmen mit dem GND Pin des ESP32 verbunden, da andernfalls die Bauteile nicht richtig funktionieren (unterschiedliche Potentiale)

2. Erste Testläufe, Problemdiagnostik und Lösungsansätze:
    - -Motoren mit großer H-Brücke und vorhandenem Batteriepack getestet.
        - -Mit einzelnen Motorn konnten hohe Drehzahlen erreicht werden.
        - -Schon mit nur Zwei Motoren gab es Leistungseinbrüche.
    - -Daraufhin wurde das Netzteil durch ein 12 Volt Netzteil ersetzt, mit dem Ergebnis, dass die Leistungseinbrüche nicht auftraten.
        - -Mit 12V enstand ein neues Problem: zu hohe Drehzahlen. Kann später analog über das ESP32 eingestellt werden, also Problem defacto gelöst.

Entsprechend wurde ein 8x AAA-Batteriepack mit einer Spannungsversorgung von 12V bestell. (In Reihe geschaltet) → erhöhung der maximal möglichen Leistung auf die im Testversuch ermittelten Werte um beobachtete Spannungseinbrüche bei multiplen Motoren zu umgehen.

3. Änderung der Pinbelegungen:
    - -Nachdem der Source Code für die Sensoren angepasst wurde, musste die Pinbelegung für die Sensoren entsprechen angepasst werden.
    - -Nur die Pins GPIO0 bis GPIO4 (PMW Pins) können für die Sensoren verwendet werden, etnsprechend wurden Motoren für das Fahrwerk angepasst (IB1→GPIO4; IB2→GPIO5).

Dabei entstand ein neues Problem:
- -Falls Sensoren und Fahrwerksmotoren über die Powerbank versorgt wurden gab es Spannungseinbrüche. Die Spannungsversorgung der Sensoren wurde direkt auf das ESP32 gelegt. Obwohl auch dieses von der Powerbank mit Strom versorgt wird hat die zwischenlegung des ESP32 dafür gesorgt, dass die Spannung stabil bleibt.

4. Weitere Testläufe, Problemdiagnostik und Lösungsansätze:
    - -Sensoren zeigen Signalglitches (SignalSprünge), auch nach angepasster Pinbelegung.
    - →Problem lag am Abstand der Sensoren zum Boden (7mm), gelöst wurde dies durch eine veringerung des Abstandes (3mm). Weitere mögliche Gründe waren die stabilität des Sensoren am Fahrzeug. Dies wurde verbessert durch tausch von M3 auf M4 Gewindestangen, wodurch auch die schwarze Linie besser erkannt werden konnte.
5. Provisorische Anbauten/Änderungen am Fahrzeug:
    - -Gestell+Platform auf das Fahrzeug gebaut, um eine weitere vertikale Ebene zu schaffen, auf welcher das größere Batteriepack untergebracht werden konnte.
    - →Gestell besthet aus M4 Gewindestangen, welche mit Muttern am bestehenden Fahrzeug fixiert wurden. Die Platform is eine reguläre Holzplatte.

Verkabelung für den Schussmechanismus wurde vorbereitet, die notwendigen Längen sind jedoch noch unbekannt und somit noch nicht gekürzt.

Design & 3D-Druck (Max Wimmer)

Erster Meilenstein:

Folgende Problematiken wurden innerhalb der ersten Entwicklungsphase (erster Meilenstein) behoben:

-Platz für eine zweite H-Brücke schaffen.
-Löcher für die Schrauben zur Befestigung am Fahrzeuggestell verkleinern.
-Modularität über Steckverschluss-System ausarbeiten.

Umsetzung:

-Bestehende H-Brücken Aufnahme kopiert und nochmals auf selbiger Fläche mit gleichen Parametern extrudiert.
-Die Löcher an den Stützen von 3.6mm auf 2mm verkleinert.
-Einkerbungen an den oberen Rändern des Halters ausgespart.

Entwicklung des modularen Steckverchluss-Systems:
(Für visuelle Darstellung der Prototypen siehe Quellen und Hilfsmittel)

Erster Prototyp:
Resultat:
Die Klammern sind nicht flexibel genug.

Zweiter Prototyp:
Änderung:
Klammer-Arme dünner.
Annahme:
Höhere Flexibilität.
Resultat:
Kleine Test-Variante funktioniert, jedoch im Bereich der benötigten Größenordnung sind die dünneren Klammer-Arme zu spröde.

Dritter Prototyp:
Änderung:
Klammer-Arme näher zur Mitte des Klammer-Körpers verschoben.
Pin in der Mitte erweitert mit Einkerbung im F-Teil.
Annahme:
Höhere Duktilität.
Resultat:
Immer noch zu spröde.

Vierter Prototyp:

Änderung:

Grundlegend überholtes Design. Klammer-Arme bereits angewinkelt. Keine toleranzen.

Annahme:

Verzicht auf Flexibilität durch neues Design ausgeglichen und mit höherer Festigkeit genug Stabilität für Steckverschluss-System gewährleistet.

Resultat:

Ausreichend zufriedenstellendes Ergebnis.

Fünfter Prototyp:

Änderung:

Steckverschluss-System verworfen und abgewandelt in ein Fügedruck-System welches dennoch die gewünschte modularität erschafft.

Annahme:

Simpleres Design schafft höhere belastbarkeit und zugfestigkeit.

Resultat:

Annahme korrekt, simpleres Design mit gleichwertigem oder besserwertigem Ergebnis.

Zweiter Meilenstein:

Im Rahmen des zweiten Meilensteins war die Aufgabe eine angemessene Umsetzung des zu entwicklenden Schussmechanismus und eventueller Ergänzung des integrierten Aufnahmemechanismus zu Designen und Drucken.

Zunächst wurde dafür ein sogenannter Spacer an das schon bestehende Design hinzugefügt. Dieser Spacer ist eine einfache, Fünf Centimeter lange Verlängerung, welche durch das innerhalb des ersten Meilensteins etnwickelte Stecksystem problemlos an das bestehende Fahrzeug angebunden werden konnte.

Im nächsten Schritt wurde der, innerhalb der Gruppe designte, Schussmechanismus provisorisch Umgesetzt. Das Design besteht aus einem Kasten in dem die Spielbälle aufgenommen werden sollen, und von diesem auch wieder heraus geschossen werden können. Dafür beherbergt der Kasten eine zentrale Achse, an welcher Vier Platten in gleichmäßigem Abstand angebracht sind. In jedem der Vier Bereiche des Kasten sollten nun jeweils ein Spielball Platz finden. Durch die Rotation der zentralen Achse wird so der Schussmechanismus mit Bällen eingespeist. Der Schussmechanismus selbst besteht aus zwei Rädern, welche von jeweils einem didizierten Motor angetrieben werden, und sich in gegensätzliche Richtungen drehen, um die Bälle ziwschen den Rädern zu Beschleunigen und somit zu schießen.
Der Aufnahmemechanismus ist sehr Zentral, da über das Kastensystem gesteuert wird wann die Bälle an den Schussmechanismus gelangen, und auch das nur maximal Drei Spielbälle gleichzeitig innerhalb des Fahrzeugs zwischengelagert werden können, konform zu den Spielregeln.

Beim ersten Prototypen ergaben sich folgende Probleme:

- -Die Wände des Kastens sind sehr fragil und brechen bei dem Versuch das ‚Rad‘ um die Zentralachse einzusetzen.
- -Der 3D-Druck ist zu ungenau, wodurch das ‚Rad‘ mit Zentralachse herausfällt oder fest steckt.

Angestrebte Lösungsversuche durch anpassung der Druck-parameter blieben erfolgslos. Eine alternative Lösung war, die Platten welche an die Zentralachse angeschlossen werden über ein neu konstruiertes Pin-System erst einzusetzen, nachdem der Kasten schon fertig konstruiert ist. Jedoch auch in diesem Fall war der Druck zu ungenau und resultierte in zu hoher Reibung (schlechte bis keine Rotation) oder zu wenig stabilität (Zentralachse fällt aus Verankerung).

Eine Notfalllösung ist bereits vorgesehen, für den Fall, dass die bestehenden Probleme mit den erdachten Mechanismen die Technische Umsetzung durch unzureichende Drucke weiterhin misslingen sollte:
In jenem Fall wird ein einfaches Schiebe-System, ähnlich zu einem Schneepflug, mit dem bestehenden Stecksystem an das Fahrzeug montiert. Die spezifikationen des ‚Pflug‘ sind weiterhin, dass die Größe den Regeln konform die maximalen Dimensionen des Fahrzeuges nicht überschreitet, und gleichzeitig maximal Drei Spielbälle aufnehmen kann.
Der ‚Schussmechanismus’ wäre in diesem Fall ausschließlich durch die Bewegung des Fahrzeugs selbst realisiert, ohne tatsächlichen Mechanismus.

Programmieren (Bastian Büld)
(Source Code unter Quellen und Hilfsmittel)

Folgende Aufgaben sollte das Programm in der ersten Entwicklungsphase (erster Meilenstein) erfüllen:

-Einbindung aller benötigten Libraries für dritt-software (z.B. Smartphone App zur Steuerung).
-Kabellose Verbindung über Smartphone mit dem Fahrzeug.
-Steuerung des Fahrzeuges hinsichtlich fahrbarkeit (beschleunigen, lenken, bremsen/halten).

https://git.ide3.de/babue001/bm3.git

Der Code enthält auch bereits potentielle Lösungen für folgende Aufgaben:

-Auslesen eines Sensors, zur erkennung eines Farbunterschiedes.
Entsprechende Reaktion auf den Sensorwert. (z.B. Stopp-Automatik).
-Ansteuerung entsprechender Motoren für Aufnahme- und Schieß-Mechanismen.

Jedoch konnten diese Funktionen noch nicht getestet werden, auf Grund der aktuellen Entwicklungsphase der teils eingeschränkten Verkabelung als auch des 3D-Drucks.

(Zweiter Meilenstein)
-Änderungen des Source Code unter Quellen und Hilfsmittel (zweiter Meilenstein)

Folgende Änderungen, Einbindungen wurden dem Source Code angefügt:

-Auslesen der Sensoren, zur erkennung eines Farbunterschied (Schwarze Linie auf Boden).
-Entsprechende Reaktion auf den Sensorwert.
-Ansteuerung entrsprechender Moroten für Aufnahme- und Schieß-Mechanismen.
(Da noch nicht vorhanden nur theoretischer Code, noch nicht getestet oder eingebunden.)

Quellen und Hilfsmittel

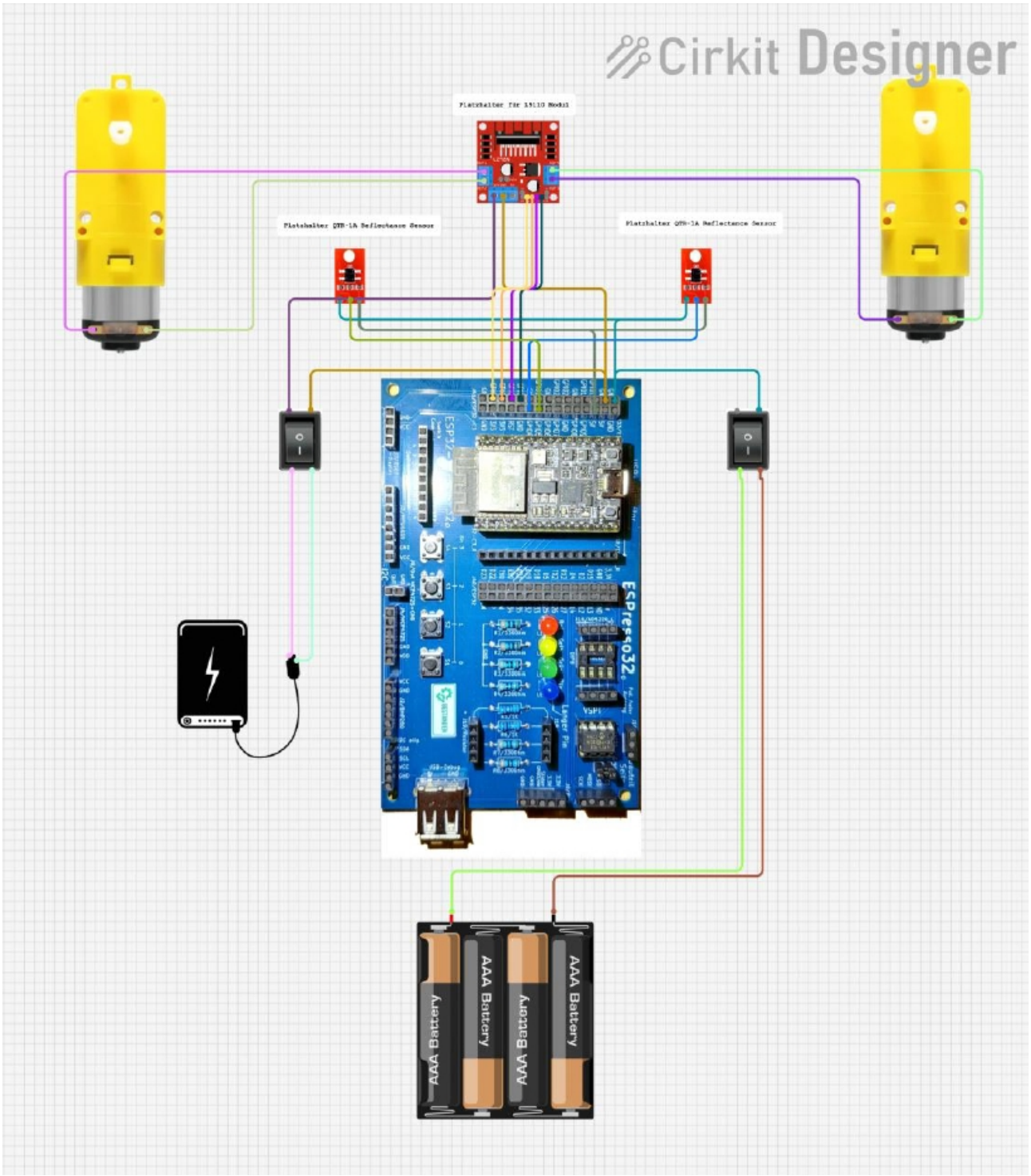
Die folgenden Softwares wurden von unserer Gruppe zur Kommunikation und Datenaustausch verwendet (Hinweis: Projektnotwendige bzw. vorgeschriebene Software, z.B. RemoteXY wird nicht gelistet):

WhatsApp (https://whatsapp.com) für allgemeine Kommunikations über Textnachrichten.
Discord (https://discord.com) für Meetings über Videoanrufe.
IDE3 (https://git.ide3.de) für Datenaustausch über ein Grupen Repository.
Cirkitdesigner (https://app.cirkitdesigner.com) zum erstellen der Schaltpläne.

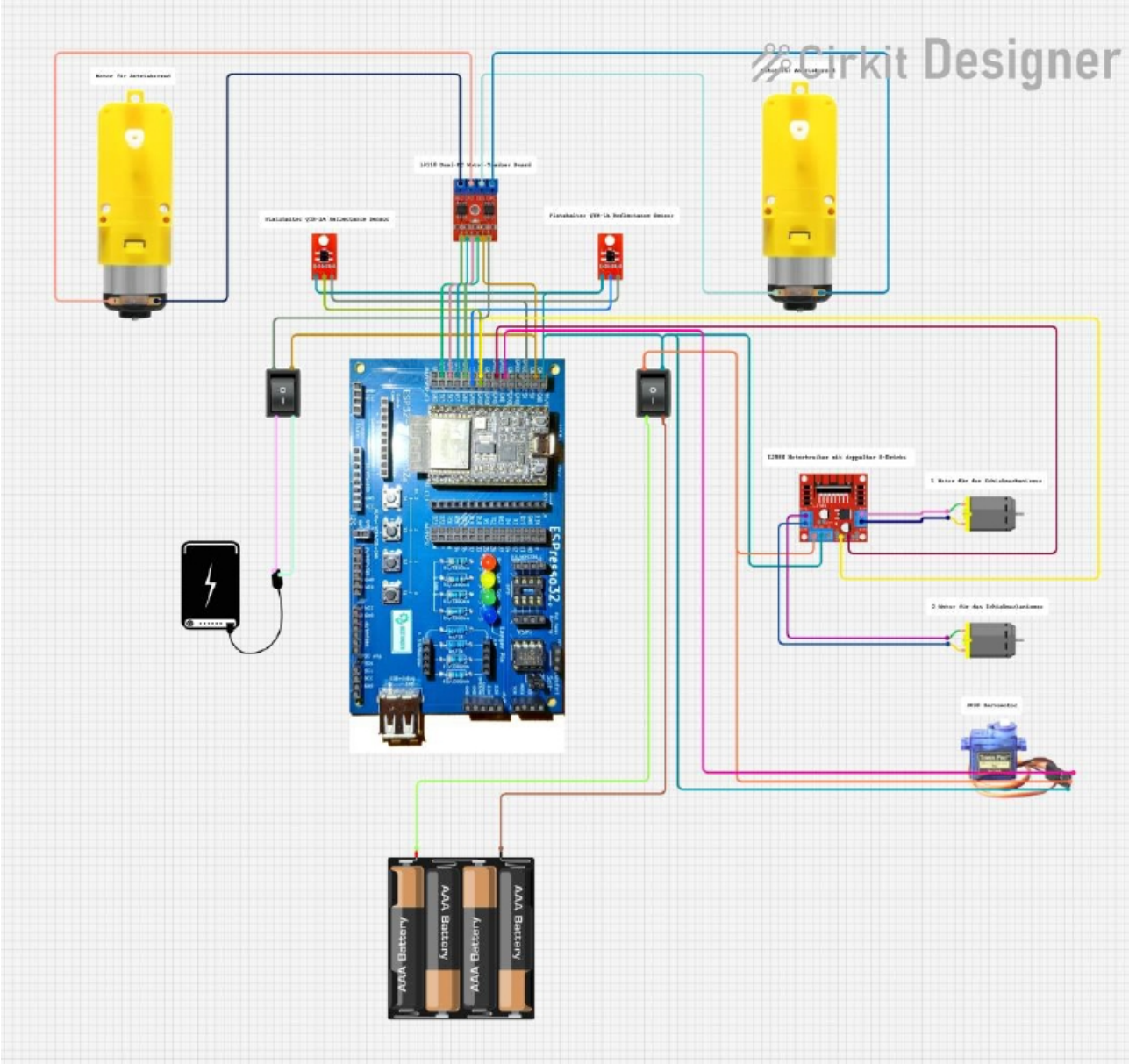
Ergebnis des ersten Gruppenmeetings via Discord (Whiteboard funktion)



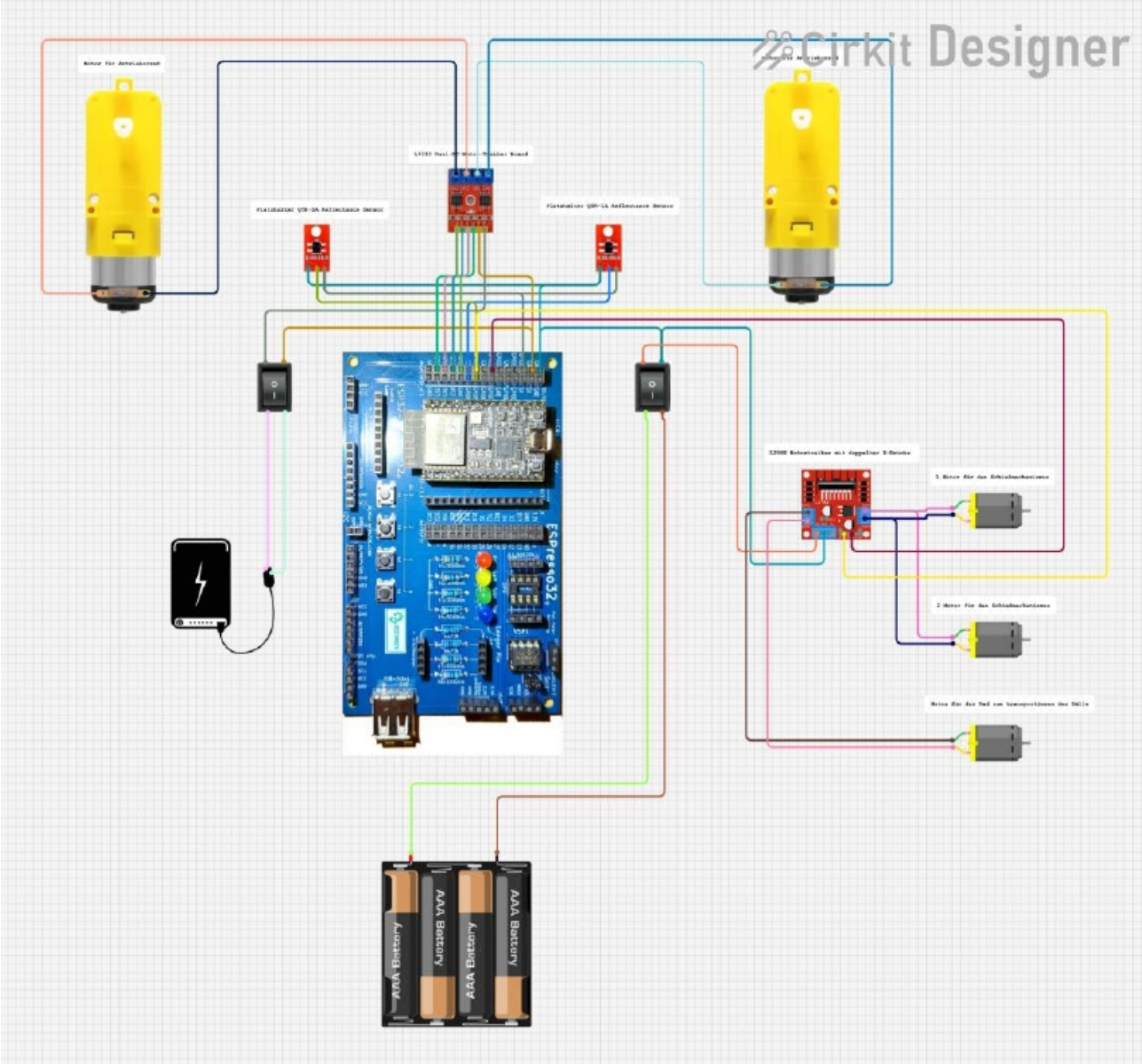
Schaltplan V1



Schaltplan V2



Schaltplan V3



Pin Belegungen (erster Meilenstein)

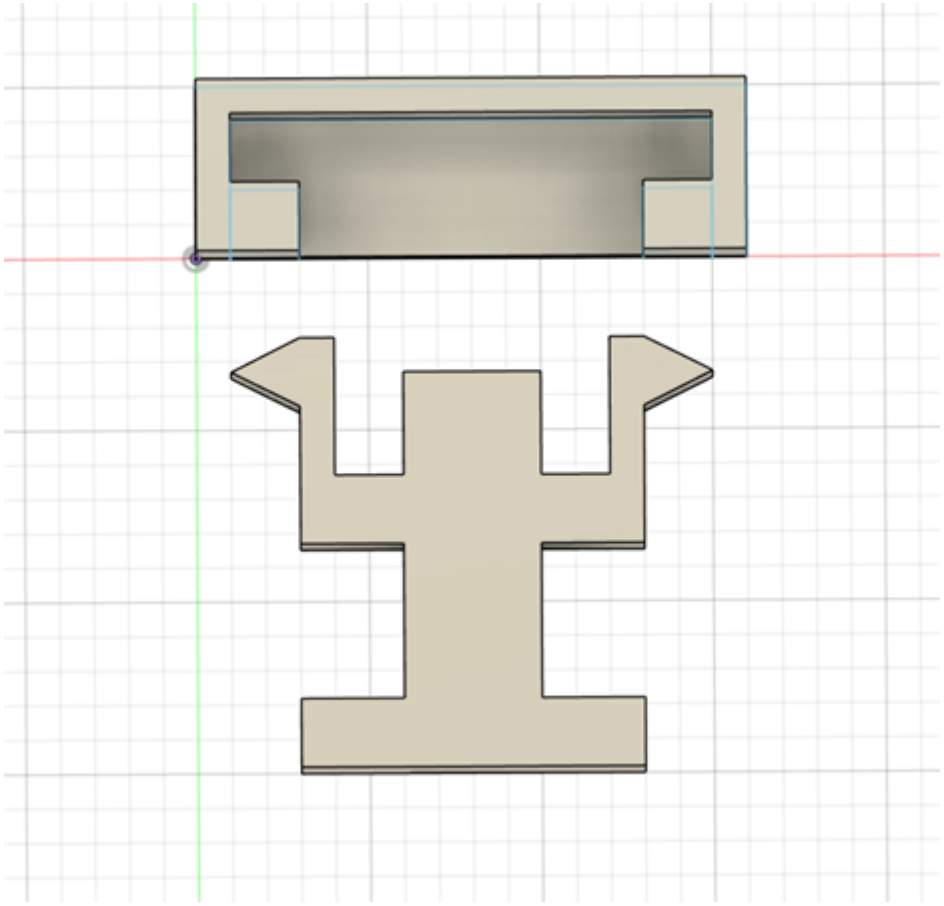
Pin Belegung L9110	ESP Pin
IB1	GPIO0
IA1	GPIO1
IB2	GPIO2
IA2	GPIO3
5V	Power Bank
GND	Power Bank

Pin Belegung QTR-1A Reflectance Sensor	ESP Pin
Sensor 1	GPIO4
Sensor 2	GPIO5
5V	Beide 5V Arduino Eingänge
GND	Beide GND Eingänge Arduino

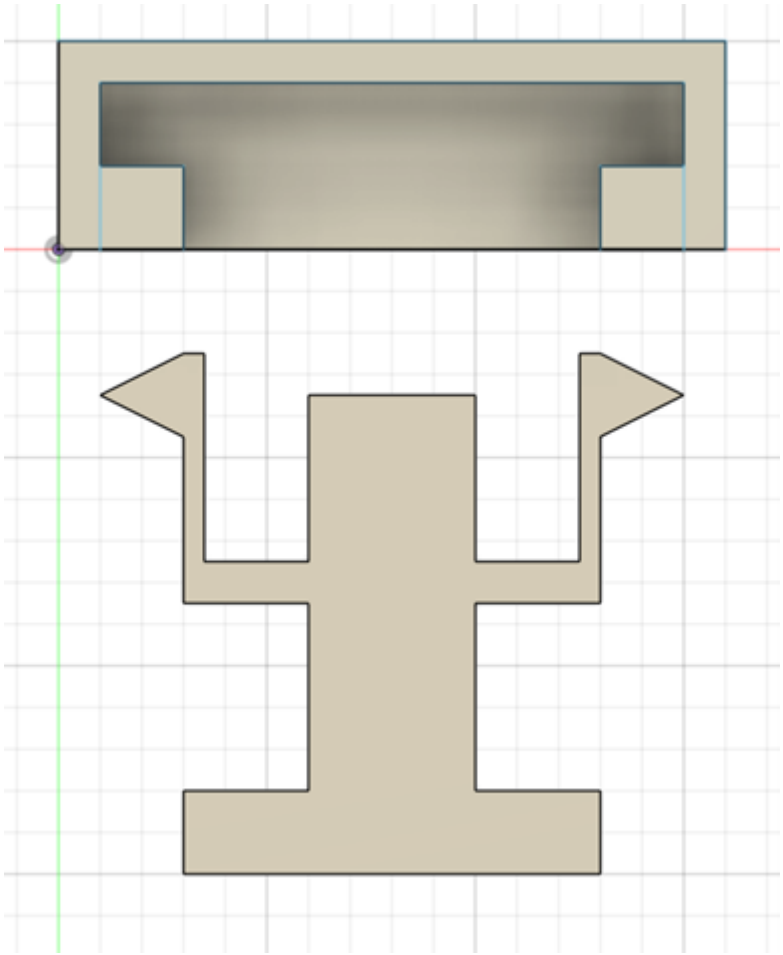
Pin Belegung LN208	ESP Pin	
IN1	GPIO10	Rad
IN2		
IN3	GPIO20	Beide Motoren für die Schleuder
IN4		
5V	Batterie	
GND	Batterie	

Pin Belegung Motoren für das Rad zum Transportieren der Bälle	ESP Pin
PWM	GPI021
5V	Batterie
GND	Batterie

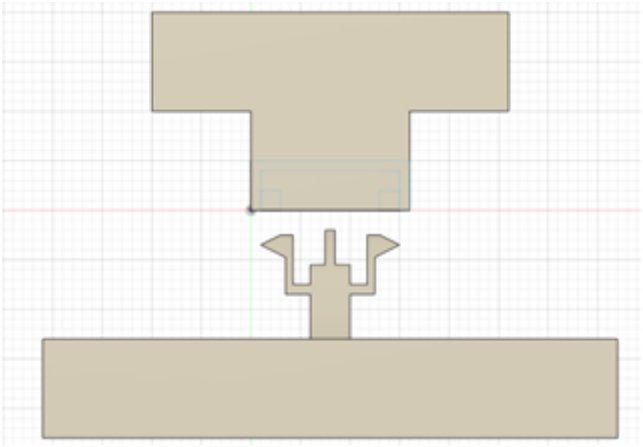
Erster Prototyp



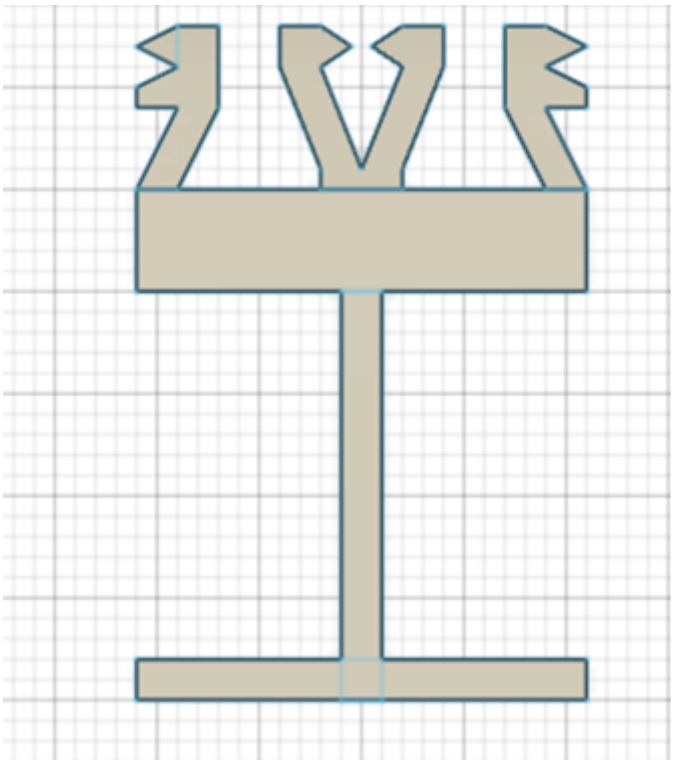
Zweiter Prototyp



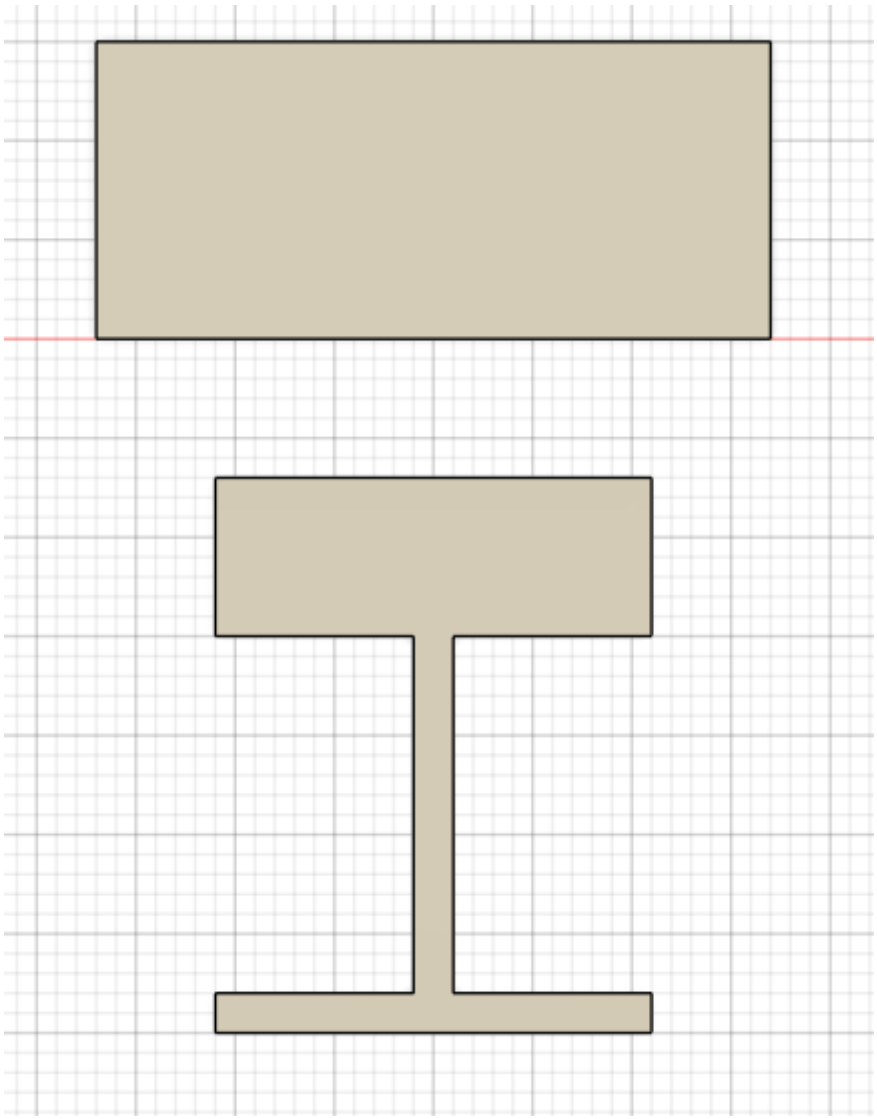
Dritter Prototyp



Vierter Prototyp



Fünfter Prototyp



Source Code des Programms (erster Meilenstein)

main.cpp 1.22 KiB

```cpp
#include <Arduino.h>
#include "bm3/peripheral/Motor.h"
#include "bm3/config/RemoteXYConfig.h"
#include "esp_task_wdt.h"
#include  "bm3/input/InputInterpreterFactory.h"
#include  "bm3/state/State.h"
#include "config/PWMInit.h"
#include "bm3/config/PwmPin.h"
#include "bm3/config/PwmSink.h"

using bm3::peripheral::Motor;
using bm3::input::InputInterpreterFactory;
using bm3::input::InputInterpreter;
using bm3::state::State;

PwmSink pwmSink;

PwmPin leftPwmA(0);
PwmPin leftPwmB(1);
PwmPin rightPwmA(2);
PwmPin rightPwmB(3);

Motor leftMotor(&leftPwmA, &leftPwmB, &pwmSink);
Motor rightMotor(&rightPwmA, &rightPwmB, &pwmSink);

void setup()
{
    RemoteXY_Init();
    Serial.begin(9600);

    PWM_init();

    // pwm pins are setup by the motor class
    leftMotor.setup();
    rightMotor.setup();
}

void loop()
{
    RemoteXY_Handler();

    if (RemoteXY.is_connected && RemoteXY.is_on)
    {
        InputInterpreter* inputInterpreter = InputInterpreterFactory::createInterpreter(
            State::DEFAULT_STATE, &leftMotor, &rightMotor);
        inputInterpreter->interpretInput(RemoteXY.vertical_slider, RemoteXY.horizontal_slider);
        pwmSink.flush(); // Synchronize all PWM updates
    }

    esp_task_wdt_reset(); // Pet the Dog
}
```

StateMachine.cpp 168 B

```cpp
#include "bm3/state/StateMachine.h"
#include "bm3/state/State.h"

using bm3::state::State;

namespace bm3 { namespace state {

// ...StateMachine implementation...

}}
```

State.h 112 B

```h
#pragma once

namespace bm3 { namespace state {

enum class State {
    DEFAULT_STATE,
    LINE_CROSSED
};

}}
```

StateMachine.h 115 B

```h
#pragma once

namespace bm3 { namespace state {

class StateMachine {
    // ...empty StateMachine class...
};

}}
```

BallShooter.cpp 158 B

```cpp
#include "bm3/peripheral/BallShooter.h"

using bm3::peripheral::BallShooter;

namespace bm3 { namespace peripheral {

// ...BallShooter implementation...

}}
```

BallShooter.h 247 B

```h
#pragma once

#include "bm3/peripheral/AbstractPeripheral.h"

using bm3::peripheral::AbstractPeripheral;

namespace bm3 { namespace peripheral {

class BallShooter {
public:
    virtual ~BallShooter() {}
    // ...other virtual methods...
};

}}
```

AbstractPeripheral.h 163 B

```h
#pragma once

namespace bm3 { namespace peripheral {

class AbstractPeripheral {
public:
    virtual void setup() = 0;
    virtual ~AbstractPeripheral() {}
};

}}
```

Motor.cpp 949 B

```cpp
#include <Arduino.h>
#include "bm3/peripheral/Motor.h"

namespace bm3 { namespace peripheral {

Motor::Motor(PwmPin* pinA, PwmPin* pinB, PwmSink* sink)
    : pinA(pinA), pinB(pinB), sink(sink) {
}

void Motor::setup() {
    Serial.println("Motor::setup");
    pinA->setup();
    pinB->setup();
}

void Motor::forward(const int speed) const
{
    if (speed < 0 || speed > 100)
    {
        Serial.println("Motor::forward: invalid speed: " + String(speed));
        return;
    }
    const int pwm = map(speed, 0, 100, 120, 255);
    sink->set(pinA, pwm);
    sink->set(pinB, 0);
}

void Motor::backward(const int speed) const{
    if (speed < 0 || speed > 100)
    {
        Serial.println("Motor::backward: invalid speed: " + String(speed));
        return;
    }
    const int pwm = map(speed, 0, 100, 120, 255);
    sink->set(pinA, 0);
    sink->set(pinB, pwm);
}

void Motor::stop() const
{
    sink->set(pinA, 0);
    sink->set(pinB, 0);
}

}}
```

Motor.h 589 B

```h
#pragma once

#include "bm3/peripheral/AbstractPeripheral.h"
#include "bm3/config/PwmPin.h"
#include "bm3/config/PwmSink.h"

using bm3::peripheral::AbstractPeripheral;

namespace bm3 { namespace peripheral {
    class Motor final : public AbstractPeripheral {
    public:
        Motor(PwmPin* pinA, PwmPin* pinB, PwmSink* sink);
        void forward(int speed) const; // speed: 0-100
        void backward(int speed) const; // speed: 0-100
        void stop() const;
        void setup() override;
    private:
        PwmPin* pinA;
        PwmPin* pinB;
        PwmSink* sink;
    };
}}

```

ReflectanceSensor.cpp 133 B

```cpp
#include "bm3/peripheral/ReflectanceSensor.h"

namespace bm3 { namespace peripheral {

// ...ReflectanceSensor implementation...

}}

```

ReflectanceSensor.h 128 B

```h
#pragma once

namespace bm3 { namespace peripheral {

class ReflectanceSensor {
  // ...empty ReflectanceSensor class...
};

}}

```

ServoBallShooter.cpp 173 B

```cpp
#include "bm3/peripheral/ServoBallShooter.h"

using bm3::peripheral::ServoBallShooter;

namespace bm3 { namespace peripheral {

// ...ServoBallShooter implementation...

}}
```

ServoBallShooter.h 218 B

```cpp
#pragma once

#include "bm3/peripheral/AbstractPeripheral.h"

using bm3::peripheral::AbstractPeripheral;

namespace bm3 { namespace peripheral {

class ServoBallShooter {
  // ...empty ServoBallShooter class...
};

}}
```

InputInterpreter.cpp 230 B

```cpp
#include "bm3/input/InputInterpreter.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;
using bm3::peripheral::BallShooter;

namespace bm3 { namespace input {

// ...InputInterpreter implementation...

}}
```

h **InputInterpreter.h** 968 B

```h
#pragma once

#include "bm3/peripheral/Motor.h"
#include "bm3/peripheral/ReflectanceSensor.h"
#include "bm3/peripheral/BallShooter.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;
using bm3::peripheral::BallShooter;

namespace bm3 { namespace input {

class InputInterpreter {
public:
    virtual void interpretInput(int slider1, int slider2) = 0;
    virtual ~InputInterpreter() = default;
protected:
    Motor* leftMotor;
    Motor* rightMotor;
    ReflectanceSensor* leftSensor;
    ReflectanceSensor* rightSensor;
    BallShooter* ballShooter;
    InputInterpreter(Motor* leftMotor,
                     Motor* rightMotor,
                     ReflectanceSensor* leftSensor,
                     ReflectanceSensor* rightSensor,
                     BallShooter* ballShooter)
        : leftMotor(leftMotor), rightMotor(rightMotor),
          leftSensor(leftSensor), rightSensor(rightSensor),
          ballShooter(ballShooter) {}
};

}}

```

InputInterpreterFactory.cpp 1.11 KiB

```cpp
#include "bm3/input/InputInterpreterFactory.h"
#include "bm3/input/SliderInputInterpreter.h"
#include "bm3/input/OverriddenInputInterpreter.h"
#include "bm3/state/State.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;
using bm3::peripheral::BallShooter;
using bm3::state::State;

namespace bm3
{
    namespace input
    {
        InputInterpreter* InputInterpreterFactory::createInterpreter(
            const State state,
            Motor* leftMotor,
            Motor* rightMotor
            // ReflectanceSensor* leftSensor,
            // ReflectanceSensor* rightSensor,
            // BallShooter* ballShooter
        )
        {
            if (state == State::DEFAULT_STATE)
            {
                // TODO remove nullprts
                return new SliderInputInterpreter(leftMotor, rightMotor, nullptr, nullptr, nullptr);
            }
            if (state == State::LINE_CROSSED)
            {
                // TODO remove nullprts
                return new OverriddenInputInterpreter(leftMotor, rightMotor, nullptr, nullptr);
            }
            return nullptr;
        }
    }
}

```

InputInterpreterFactory.h 673 B

```cpp
#pragma once

#include "bm3/input/InputInterpreter.h"
#include "bm3/peripheral/Motor.h"
#include "bm3/peripheral/ReflectanceSensor.h"
#include "bm3/peripheral/BallShooter.h"
#include "bm3/state/State.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;
using bm3::peripheral::BallShooter;
using bm3::state::State;

namespace bm3 { namespace input {

class InputInterpreterFactory {
public:
    static InputInterpreter* createInterpreter(
        State state,
        Motor* leftMotor,
        Motor* rightMotor
        // ReflectanceSensor* leftSensor,
        // ReflectanceSensor* rightSensor,
        // BallShooter* ballShooter
        );
};

}}

```

JoystickInputInterpreter.cpp 246 B

```cpp
#include "bm3/input/JoystickInputInterpreter.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;
using bm3::peripheral::BallShooter;

namespace bm3 { namespace input {

// ...JoystickInputInterpreter implementation...

}}

```

JoystickInputInterpreter.h 367 B

```cpp
#pragma once

#include "bm3/peripheral/Motor.h"
#include "bm3/peripheral/ReflectanceSensor.h"
#include "bm3/peripheral/BallShooter.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;
using bm3::peripheral::BallShooter;

namespace bm3 { namespace input {

class JoystickInputInterpreter {
  // ...empty JoystickInputInterpreter class...
};

}}

```

OverriddenInputInterpreter.cpp 212 B

```cpp
#include "bm3/input/OverriddenInputInterpreter.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;

namespace bm3 { namespace input {
// ...OverriddenInputInterpreter implementation...
}}

```

h **OverriddenInputInterpreter.h** 836 B

```h
#pragma once

#include "bm3/input/InputInterpreter.h"
#include "bm3/peripheral/Motor.h"
#include "bm3/peripheral/ReflectanceSensor.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;

namespace bm3 { namespace input {

class OverriddenInputInterpreter : public InputInterpreter {
public:
    OverriddenInputInterpreter(Motor* leftMotor,
                               Motor* rightMotor,
                               ReflectanceSensor* leftSensor,
                               ReflectanceSensor* rightSensor)
        : InputInterpreter(leftMotor, rightMotor, leftSensor, rightSensor, nullptr) {}
    void interpretInput(int slider1, int slider2) override {
        // Example: stop both motors when overridden
        if (leftMotor) leftMotor->stop();
        if (rightMotor) rightMotor->stop();
    }
};

}}

```

SliderInputInterpreter.cpp 3.21 KiB

```cpp
#include "bm3/input/SliderInputInterpreter.h"

#include <HardwareSerial.h>

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;
using bm3::peripheral::BallShooter;

namespace bm3
{
    namespace input
    {
        void SliderInputInterpreter::interpretInput(const int vertical_slider, const int horizontal_slider)
        {
            if (100 < vertical_slider || vertical_slider < -100)
            {
                Serial.println("Vertical Slider out of Bounds: " + String(vertical_slider));
                return;
            }
            if (100 < horizontal_slider || horizontal_slider < -100)
            {
                Serial.println("Horizontal Slider out of Bounds: " + String(horizontal_slider));
                return;
            }

            const bool backwards = vertical_slider < 0;
            const bool forwards = vertical_slider > 0;
            const bool standstill = !forwards && !backwards;

            const bool left = horizontal_slider < 0;
            const bool right = horizontal_slider > 0;
            const bool straight = !left && !right;

            const int speed = std::abs(vertical_slider);
            const int tilt = std::abs(horizontal_slider) * 0.93 * speed / 100.0; // max tilt is 75% of speed

            const int rotation = std::abs(horizontal_slider) * 0.5; // rotation speed when standstill would be to fast at max

            int speed_left = 0;
            int speed_right = 0;
            if (left)
            {
                speed_left = speed;
                speed_right = speed - tilt;
            }
            else if (right)
            {
                speed_left = speed - tilt;
                speed_right = speed;
            }
            else if (straight)
            {
                speed_left = speed;
                speed_right = speed;
            }

            if (forwards)
            {
                Serial.println(
                    "Left Motor forward: " + String(speed_left) + ", " + "Right Motor forward: " + String(speed_right));
                leftMotor->forward(speed_left);
                rightMotor->forward(speed_right);
            }
            else if (backwards)
            {
                Serial.println(
                    "Left Motor backward: " + String(speed_left) + ", " + "Right Motor backward: " +
                    String(speed_right));
                leftMotor->backward(speed_left);
                rightMotor->backward(speed_right);
            }
            else if (standstill && right)
            {
                Serial.println("Left Motor backward: " + String(tilt) + ", " + "Right Motor forward: " + String(tilt));
                leftMotor->backward(rotation);
                rightMotor->forward(rotation);
            }
            else if (standstill && left)
            {
                Serial.println("Left Motor forward: " + String(tilt) + ", " + "Right Motor backward: " + String(tilt));
                leftMotor->forward(rotation);
                rightMotor->backward(rotation);
            }
            else if (standstill && straight)
            {
                Serial.println("Both Motors stop");
                leftMotor->stop();
                rightMotor->stop();
            }
        }
    }
}
```

h **SliderInputInterpreter.h** 890 B

```h
#pragma once

#include "bm3/input/InputInterpreter.h"
#include "bm3/peripheral/Motor.h"
#include "bm3/peripheral/ReflectanceSensor.h"
#include "bm3/peripheral/BallShooter.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;
using bm3::peripheral::BallShooter;

namespace bm3
{
    namespace input
    {
        class SliderInputInterpreter final : public InputInterpreter
        {
        public:
            SliderInputInterpreter(
                Motor* leftMotor,
                Motor* rightMotor,
                ReflectanceSensor* leftSensor,
                ReflectanceSensor* rightSensor,
                BallShooter* ballShooter
            ) : InputInterpreter(leftMotor, rightMotor, leftSensor, rightSensor, ballShooter)
            {
            }

            void interpretInput(int vertical_slider, int horizontal_slider) override;
        };
    }
}

```

h **PWMInit.h** 192 B

```h
#pragma once
#include "ESP32PWM.h"

inline void PWM_init() {
    ESP32PWM::allocateTimer(0);
    ESP32PWM::allocateTimer(1);
    ESP32PWM::allocateTimer(2);
    ESP32PWM::allocateTimer(3);
}


```

h **PwmPin.h** 300 B

```h
#pragma once

#include "ESP32PWM.h"

class PwmPin
{
public:
    explicit PwmPin(const int pin) : pin_(pin) {}

    void setup() {
        pwm_.attachPin(pin_, 1000, 8);
    }

    void analogWrite(const uint8_t value) {
        pwm_.write(value);
    }

private:
    int pin_;
    ESP32PWM pwm_;
};


```

h **PwmSink.h** 843 B

```h
#pragma once
#include <vector>
#include <algorithm>
#include <random>
#include "bm3/config/PwmPin.h"

class PwmSink {
public:
    struct Entry {
        PwmPin* pin;
        int value;
    };

    void set(PwmPin* pin, int value) {
        for (auto& entry : entries_) {
            if (entry.pin == pin) {
                entry.value = value;
                return;
            }
        }
        entries_.push_back({pin, value});
    }

    void flush() {
        // Randomize the order of entries before flushing
        static std::random_device rd;
        static std::mt19937 g(rd());
        std::shuffle(entries_.begin(), entries_.end(), g);
        for (auto& entry : entries_) {
            if (entry.pin) entry.pin->analogWrite(entry.value);
        }
        entries_.clear();
    }

private:
    std::vector<Entry> entries_;
};

```

h **RemoteXYConfig.h** 1.07 KiB

```h
#pragma once

#pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wglobal-constructors"
#pragma clang diagnostic ignored "-Wmissing-variable-declarations"
#pragma clang diagnostic ignored "-Wreserved-identifier"
#pragma clang diagnostic ignored "-Wpragma-pack"

// NOLINTNEXTLINE(cppcoreguidelines-avoid-reserved-identifier, bugprone-reserved-identifier)
#define REMOTEXY_MODE__ESP32CORE_BLE
#define REMOTEXY_BLUETOOTH_NAME "RemoteXY"

#include <BLEDevice.h>
#include <RemoteXY.h>

#pragma pack(push, 1)
uint8_t RemoteXY_CONF[] =   // 62 bytes
  { 255,4,0,0,0,55,0,19,0,0,0,0,24,1,106,200,1,1,4,0,
  2,64,6,33,7,0,2,26,31,31,79,78,0,79,70,70,0,4,37,72,
  62,11,176,2,26,4,18,44,12,69,48,2,26,1,55,96,27,27,0,2,
  31,0 };

struct {
    uint8_t is_on; // =1 if switch ON and =0 if OFF, from 0 to 1
    int8_t horizontal_slider; // from -100 to 100
    int8_t vertical_slider; // from -100 to 100
    uint8_t button_01; // =1 if button pressed, else =0, from 0 to 1
    uint8_t is_connected;  // =1 if wire connected, else =0
} RemoteXY;
#pragma pack(pop)

#pragma clang diagnostic pop

```

Quellen und Hilfsmittel (zweiter Meilenstein):

Änderungen bei der Pinbelegung (rot hinterlegt):

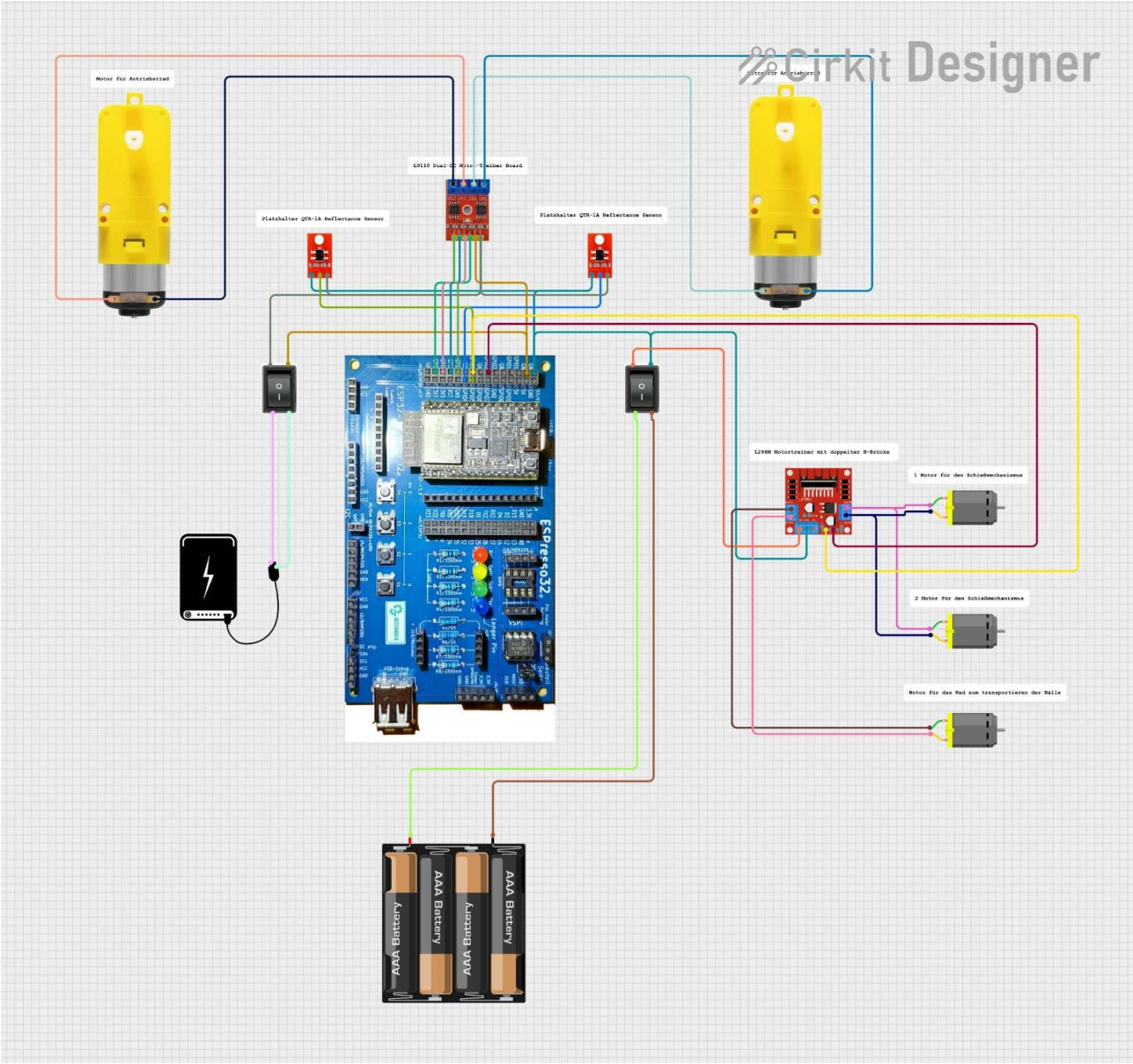
Pin Belegung L9110	ESP Pin
IB1	GPIO4
IA1	GPIO5
IB2	GPIO2
IA2	GPIO3
5V	Power Bank
GND	ESP32

Pin Belegung QTR-1A Reflectance Sensor	ESP Pin
Sensor 1	GPIO0
Sensor 2	GPIO1
5V	Beide 5V Arduino Eingänge
GND	ESP32

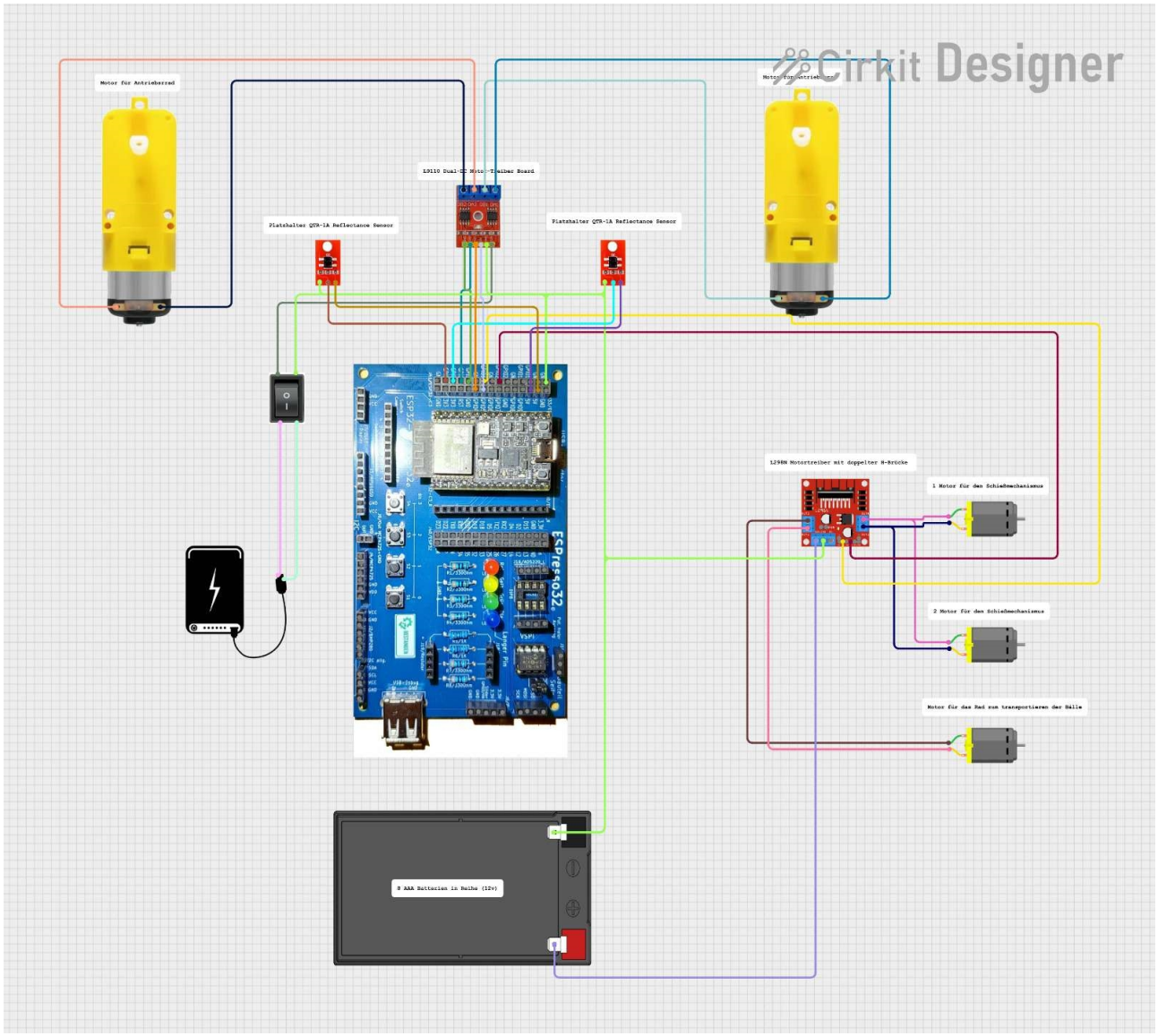
Pin Belegung LN208	ESP Pin	
IN1	GPI010	Rad
IN2		
IN3	GPIO20	Beide Motoren für die Schleuder
IN4		
5V	Batterie	
GND	ESP32	

Pin Belegung Motoren für das Rad zum Transportieren der Bälle	ESP Pin
PWM	GPI021
5V	Batterie
GND	ESP32

Schaltpläne (zweiter Meilenstein)
Schaltplan V4:



Schaltplan V5:



Änderungen und Ergänzungen des Source Code (zweiter Meilenstein):

README.md feat: wip 1 week ag

README.md

This directory is intended for project header files.

A header file is a file containing C declarations and macro definitions to be shared between several project source files. You request the use of a header file in your project source file (C, C++, etc) located in `src` folder by including it, with the C preprocessing directive `#include'.

```
#include "header.h"

int main (void)
{
 ...
}
```

Including a header file produces the same results as copying the header file into each source file that needs it. Such copying would be time-consuming and error-prone. With a header file, the related declarations appear in only one place. If they need to be changed, they can be changed in one place, and programs that include the header file will automatically use the new version when next recompiled. The header file eliminates the labor of finding and changing all the copies as well as the risk that a failure to find one copy will result in inconsistencies within a program.

In C, the usual convention is to give header files names that end with `.h'. It is most portable to use only letters, digits, dashes, and underscores in header file names, and at most one dot.

Read more about using header files in official GCC documentation:

- Include Syntax
- Include Operation
- Once-Only Headers
- Computed Includes

https://gcc.gnu.org/onlinedocs/cpp/Header-Files.html

README.md feat: wip

README.md

This directory is intended for project specific (private) libraries. PlatformIO will compile them to static libraries and link into executable file.

The source code of each library should be placed in an own separate directory ("lib/your_library_name/[here are source files]").

For example, see a structure of the following two libraries `Foo` and `Bar`:

|--lib | | |--Bar | | |--docs | | |--examples | | |--src | | |- Bar.c | | |- Bar.h | | |- library.json (optional, custom build options, etc)
https://docs.platformio.org/page/librarymanager/config.html | | |--Foo | | |- Foo.c | | |- Foo.h | | | |- README --> THIS FILE | |- platformio.ini |--src |- main.c

and a contents of `src/main.c`:

```
#include <Foo.h>
#include <Bar.h>

int main (void)
{
  ...
}
```

PlatformIO Library Dependency Finder will find automatically dependent libraries scanning project source files.

More information about PlatformIO Library Dependency Finder

- https://docs.platformio.org/page/librarymanager/ldf.html

OverriddenInputInterpreter.cpp 672 B

```cpp
#include "bm3/input/OverriddenInputInterpreter.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;

namespace bm3
{
    namespace input
    {
        // ...OverriddenInputInterpreter implementation...
        void OverriddenInputInterpreter::interpretInput(int slider1, int slider2)
        {
            Serial.println("OverriddenInputInterpreter::interpretInput");
            // Example: stop both motors when overridden
            // if (leftMotor) leftMotor->backward(50);
            // if (rightMotor) rightMotor->backward(50);
            if (leftMotor) leftMotor->stop();
            if (rightMotor) rightMotor->stop();
        }
```

InputInterpreterFactory.cpp 1.04 KiB

```cpp
#include "bm3/input/InputInterpreterFactory.h"
#include "bm3/input/SliderInputInterpreter.h"
#include "bm3/input/OverriddenInputInterpreter.h"
#include "bm3/state/State.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;
using bm3::peripheral::BallShooter;
using bm3::state::State;

namespace bm3
{
    namespace input
    {
        InputInterpreter* InputInterpreterFactory::createInterpreter(
            const State state,
            Motor* leftMotor,
            Motor* rightMotor
            // ReflectanceSensor* leftSensor,
            // ReflectanceSensor* rightSensor,
            // BallShooter* ballShooter
        )
        {
            if (state == State::DEFAULT_STATE)
            {
                return new SliderInputInterpreter(leftMotor, rightMotor, nullptr, nullptr, nullptr);
            }
            if (state == State::LINE_CROSSED)
            {
                return new OverriddenInputInterpreter(leftMotor, rightMotor, nullptr, nullptr, nullptr);
            }
            return nullptr;
        }
    }
}
```

h **OverriddenInputInterpreter.h** 903 B

```h
#pragma once

#include "bm3/input/InputInterpreter.h"
#include "bm3/peripheral/Motor.h"
#include "bm3/peripheral/ReflectanceSensor.h"

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;

namespace bm3
{
    namespace input
    {
        class OverriddenInputInterpreter final : public InputInterpreter
        {
        public:
            OverriddenInputInterpreter(Motor* leftMotor,
                                       Motor* rightMotor,
                                       ReflectanceSensor* leftSensor,
                                       ReflectanceSensor* rightSensor,
                                               BallShooter* ballShooter
            )
                : InputInterpreter(leftMotor, rightMotor, leftSensor, rightSensor, ballShooter)
            {
            }

            void interpretInput(int slider1, int slider2) override;
        };
    }
}
```

SliderInputInterpreter.cpp 3.31 KiB

```cpp
#include "bm3/input/SliderInputInterpreter.h"

#include <HardwareSerial.h>

using bm3::peripheral::Motor;
using bm3::peripheral::ReflectanceSensor;
using bm3::peripheral::BallShooter;

namespace bm3
{
    namespace input
    {
        void SliderInputInterpreter::interpretInput(const int vertical_slider, const int horizontal_slider)
        {
            Serial.println("SliderInputInterpreter::interpretInput");
            if (100 < vertical_slider || vertical_slider < -100)
            {
                Serial.println("Vertical Slider out of Bounds: " + String(vertical_slider));
                return;
            }
            if (100 < horizontal_slider || horizontal_slider < -100)
            {
                Serial.println("Horizontal Slider out of Bounds: " + String(horizontal_slider));
                return;
            }

            const bool backwards = vertical_slider < 0;
            const bool forwards = vertical_slider > 0;
            const bool standstill = !forwards && !backwards;

            const bool left = horizontal_slider < 0;
            const bool right = horizontal_slider > 0;
            const bool straight = !left && !right;

            const int speed = std::abs(vertical_slider);
            const int tilt = static_cast<int>(std::abs(horizontal_slider) * 0.93 * speed / 100.0); // max tilt is 75% of speed

            const int rotation = static_cast<int>(std::abs(horizontal_slider) * 0.5); // rotation speed when standstill would be to fast at max

            int speed_left = 0;
            int speed_right = 0;
            if (left)
            {
                speed_left = speed;
                speed_right = speed - tilt;
            }
            else if (right)
            {
                speed_left = speed - tilt;
                speed_right = speed;
            }
            else if (straight)
            {
                speed_left = speed;
                speed_right = speed;
            }

            if (forwards)
            {
                Serial.println(
                    "Left Motor forward: " + String(speed_left) + ", " + "Right Motor forward: " + String(speed_right));
                leftMotor->forward(speed_left);
                rightMotor->forward(speed_right);
            }
            else if (backwards)
            {
                Serial.println(
                    "Left Motor backward: " + String(speed_left) + ", " + "Right Motor backward: " +
                    String(speed_right));
                leftMotor->backward(speed_left);
                rightMotor->backward(speed_right);
            }
            else if (standstill && right)
            {
                Serial.println("Left Motor backward: " + String(tilt) + ", " + "Right Motor forward: " + String(tilt));
                leftMotor->backward(rotation);
                rightMotor->forward(rotation);
```

```
                leftMotor->backward(rotation);
                rightMotor->forward(rotation);
            }
            else if (standstill && left)
            {
                Serial.println("Left Motor forward: " + String(tilt) + ", " + "Right Motor backward: " + String(tilt));
                leftMotor->backward(rotation);
                rightMotor->forward(rotation);
            }
            else if (standstill && straight)
            {
                Serial.println("Both Motors stop");
                leftMotor->stop();
                rightMotor->stop();
            }
        }
    }
}

```

Motor.cpp 1.23 KiB

```cpp
#include <Arduino.h>
#include "bm3/peripheral/Motor.h"

namespace bm3
{
    namespace peripheral
    {
        Motor::Motor(PwmPin* pinA, PwmPin* pinB, PwmSink* sink)
            : pinA(pinA), pinB(pinB), sink(sink)
        {
        }

        void Motor::setup()
        {
            Serial.println("Motor::setup");
            pinA->setup();
            pinB->setup();
        }

        void Motor::forward(const int speed) const
        {
            if (speed < 0 || speed > 100)
            {
                Serial.println("Motor::forward: invalid speed: " + String(speed));
                return;
            }
            const int pwm = map(speed, 0, 100, 120, 255);
            sink->set(pinA, pwm);
            sink->set(pinB, 0);
        }

        void Motor::backward(const int speed) const
        {
            if (speed < 0 || speed > 100)
            {
                Serial.println("Motor::backward: invalid speed: " + String(speed));
                return;
            }
            const int pwm = map(speed, 0, 100, 120, 255);
            sink->set(pinA, 0);
            sink->set(pinB, pwm);
        }

        void Motor::stop() const
        {
            sink->set(pinA, 0);
            sink->set(pinB, 0);
        }
    }
}
```

ReflectanceSensor.cpp 1001 B

```cpp
#include "bm3/peripheral/ReflectanceSensor.h"

#include <esp32-hal-adc.h>
#include <HardwareSerial.h>

namespace bm3
{
    namespace peripheral
    {
        ReflectanceSensor::ReflectanceSensor(const int pin)
        {
            readPinNumber = pin;
            reflectanceValue = analogRead(readPinNumber);
        }

        void ReflectanceSensor::update()
        {
            reflectanceValue = analogRead(readPinNumber);
            Serial.println("ReflectanceSensor::update " + String(reflectanceValue) + " isDark = " + this->isDark() + " isLight = " + this->isLight());
        }

        bool ReflectanceSensor::isDark() const
        {
            return reflectanceValue < 800;
        }

        bool ReflectanceSensor::isLight() const
        {
            return reflectanceValue >= 800;
        }

        void ReflectanceSensor::setup()
        {
            Serial.println("ReflectanceSensor::setup");
            // no need for pin mode as it is alread correct
        }
    }
}
```

h **ReflectanceSensor.h** 523 B

```h
#pragma once
#include <esp_private/esp_wifi_types_private.h>

#include "AbstractPeripheral.h"

namespace bm3
{
    namespace peripheral
    {
        class ReflectanceSensor : public AbstractPeripheral
        {
        public:
            explicit ReflectanceSensor(int readPinNumber);
            void setup() override;
            bool isDark() const;
            bool isLight() const;
            void update();

        private:
            double reflectanceValue;
            int readPinNumber{};
        };
    }
}

```

StateMachine.cpp 1.52 KiB

```cpp
#include "bm3/state/StateMachine.h"

#include <utility>
#include "bm3/state/State.h"

using bm3::state::State;

namespace bm3
{
    namespace state
    {
        StateMachine::StateMachine(State initialState, ReflectanceSensor leftReflectanceSensor,
                                   ReflectanceSensor rightReflectanceSensor) :
            leftReflectanceSensor(std::move(leftReflectanceSensor)),
            rightReflectanceSensor(std::move(rightReflectanceSensor))
        {
            state = initialState;
            lineCrossedAt = millis();
        }

        void StateMachine::update()
        {
            leftReflectanceSensor.update();
            rightReflectanceSensor.update();

            if (this->state == State::LINE_CROSSED)
            {
                unsigned long elapsed = millis() - lineCrossedAt;
                if (elapsed >= 2000)
                {
                    Serial.println("2000ms elapsed, entering Default State");
                    this->state = State::DEFAULT_STATE;
                }
            }

            if (this->state == State::DEFAULT_STATE)
            {
                if (leftReflectanceSensor.isDark() || rightReflectanceSensor.isDark())
                {
                    this->state = State::LINE_CROSSED;
                    this->lineCrossedAt = millis();
                    Serial.println("Line crossed, entering line crossed state");
                }
            }
        }

        State StateMachine::getState() const
        {
            return this->state;
        }
    }
}
```

h **StateMachine.h** 620 B

```h
#pragma once
#include "State.h"
#include "bm3/input/InputInterpreterFactory.h"

namespace bm3
{
    namespace state
    {
        class StateMachine
        {
        public:
            explicit StateMachine(State initialState, ReflectanceSensor leftReflectanceSensor,
                                  ReflectanceSensor rightReflectanceSensor);
            void update();
            State getState() const;
        private:
            State state;
            unsigned long lineCrossedAt;

            ReflectanceSensor leftReflectanceSensor;
            ReflectanceSensor rightReflectanceSensor;
        };
    }
}

```

main.cpp 1.53 KiB

```cpp
#include <Arduino.h>
#include <ios>

#include "bm3/peripheral/Motor.h"
#include "bm3/config/RemoteXYConfig.h"
#include "esp_task_wdt.h"
#include  "bm3/input/InputInterpreterFactory.h"
#include  "bm3/state/State.h"
#include "config/PWMInit.h"
#include "bm3/config/PwmPin.h"
#include "bm3/config/PwmSink.h"
#include "state/StateMachine.h"

using bm3::peripheral::Motor;
using bm3::input::InputInterpreterFactory;
using bm3::input::InputInterpreter;
using bm3::state::State;
using bm3::state::StateMachine;

PwmSink pwmSink;

PwmPin rightPwmA(5);
PwmPin rightPwmB(4);
PwmPin leftPwmA(3);
PwmPin leftPwmB(2);

Motor leftMotor(&leftPwmA, &leftPwmB, &pwmSink);
Motor rightMotor(&rightPwmA, &rightPwmB, &pwmSink);

ReflectanceSensor leftSensor(1);
ReflectanceSensor rightSensor(0);

StateMachine stateMachine(State::DEFAULT_STATE, leftSensor, rightSensor);

void setup()
{
    RemoteXY_Init();
    Serial.begin(9600);
```

```
34
35  void setup()
36  {
37      RemoteXY_Init();
38      Serial.begin(9600);
39
40      PWM_init();
41
42      // pwm pins are setup by the motor class
43      leftMotor.setup();
44      rightMotor.setup();
45
46      leftSensor.setup();
47      rightSensor.setup();
48  }
49
50  void loop()
51  {
52      RemoteXY_delay(1000);
53      RemoteXY_Handler();
54
55      // if (RemoteXY.is_connected && RemoteXY.is_on)
56      // {
57      stateMachine.update();
58      InputInterpreter* inputInterpreter = InputInterpreterFactory::createInterpreter(
59          stateMachine.getState(), &leftMotor, &rightMotor);
60      inputInterpreter->interpretInput(RemoteXY.vertical_slider, RemoteXY.horizontal_slider);
61      pwmSink.flush(); // Synchronize all PWM updates
62      // }
63
64      esp_task_wdt_reset(); // Pet the Dog
65  }
```